Evidence-Verified LLM Agents for Safe Backend Incident Remediation

Yuxuan Zheng
University of Michigan
Ann Arbor, USA
yxzheng@umich.edu

Jingfeng Zhou
University of Southern California
Los Angeles, USA
xavier.zhou.ifcen@gmail.com

Rong Hu
Carnegie Mellon University
Pittsburgh, USA
rhoung99@gmail.com

Ruochen Fang*
Carnegie Mellon University
Mountain View, USA
david.fangruochen@gmail.com

2026 3rd International Conference on Image Processing and Artificial Intelligence (ICIPAI) | 979-8-3315-8374-3/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICIPAI70034.2026.11605740

***Abstract*—Large Language Model (LLM) agents have shown promise in automating cloud incident management. However, existing approaches often allow LLMs to execute repair actions based on insufficiently validated reasoning, leading to erroneous fixes. We propose VeriOps-Agent, a framework that decouples hypothesis generation from remediation execution through an Evidence Gate mechanism. The LLM generates candidate root causes and fix plans, which are scored along three dimensions: evidence coverage, operation safety, and rollback feasibility. Only when the composite gate score exceeds a calibrated threshold is autonomous execution permitted; otherwise, the incident is escalated to human operators. On Kubernetes microservice fault scenarios covering six fault categories, VeriOps-Agent reduces the wrong fix rate by approximately 50–55% compared to direct LLM agents and unverified autonomous agents, while maintaining competitive root cause accuracy with moderate MTTR overhead.**

Keywords: AIOps, LLM Agents, Root Cause Analysis, Incident Remediation, Evidence Verification, Kubernetes

I. Introduction

The growing scale and complexity of cloud-native microservice architectures have made incident management one of the most resource-intensive aspects of site reliability engineering. Production incidents ranging from pod crashes and CPU throttling to configuration drifts demand rapid root cause analysis (RCA) and timely remediation to minimize customer impact [1]. Traditional approaches rely heavily on human on-call engineers who manually inspect logs, metrics, and traces—a process that is slow, error-prone, and increasingly unsustainable as system complexity grows [2].

Recent advances in Large Language Models (LLMs) have catalyzed a new paradigm for incident management. Systems such as RCAgent [3] and Xpert [4] leverage LLMs to automate RCA by analyzing diagnostic information and generating root cause predictions. The ReAct framework [5] enables LLM agents to interleave reasoning with tool-augmented actions, and retrieval-augmented generation (RAG) [6] has been applied to enrich LLM context with historical incident data.

However, a critical gap persists between *diagnosis* and *safe remediation*. When LLM agents execute remediation actions directly—such as restarting services, scaling resources, or rolling back deployments—the risk of erroneous fixes becomes substantial due to hallucination [7]. Recent benchmarks highlight this challenge: ITBench [8] reports that state-of-the-art LLM agents resolve only 11.4% of SRE scenarios, OpenRCA [9] shows only 11.34% accuracy on complex root cause identification, and AIOpsLab [10] reveals that LLM agents frequently take harmful actions during fault remediation.

To address this gap, we propose VeriOps-Agent, a framework built on the principle that *evidence must precede action*. VeriOps-Agent introduces an *Evidence Gate*—a verification layer that scores each candidate fix along three safety dimensions before permitting execution: (1) evidence coverage, measuring how well the diagnosis is supported by observable telemetry; (2) operation safety, quantifying the blast radius and data-loss risk; and (3) rollback feasibility, assessing whether the system state can be restored if the fix fails. Only when the composite score exceeds a threshold is autonomous remediation allowed.

Our contributions are: (1) an evidence-verified agent architecture that separates RCA hypothesis generation from remediation execution through a quantitative evidence gate; (2) three safety-oriented scoring dimensions and a composite gate function providing a tunable trade-off between automation and safety; and (3) evaluation on Kubernetes fault scenarios demonstrating significant reductions in wrong fix rates compared to baselines.

II. Methodological Foundation

Evidence-grounded reasoning provides the first methodological basis of the proposed framework. LLM-driven diagnostic agents establish the feasibility of transforming heterogeneous operational observations into structured fault hypotheses and actionable explanations. Self-refining reasoning mechanisms further demonstrate the value of iterative hypothesis refinement for improving diagnostic consistency in complex software environments [11], [12]. Structured reasoning methodologies emphasize explicit associations between inferred conclusions and supporting evidence, enabling reasoning processes to remain traceable and verifiable throughout decision generation [13], [15]. Complementary advances in explainable representation learning and semantic retrieval integration further strengthen

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:54 UTC from IEEE Xplore. Restrictions apply.

the interpretability and consistency of reasoning outcomes by linking generated conclusions with coherent semantic evidence structures [14], [16]. Runtime-aware retrieval systems extend these principles into cloud operational environments by continuously aligning analytical reasoning with evolving observability signals and historical operational knowledge [17]. Collectively, these methodological developments establish the conceptual foundation for the evidence coverage dimension of VeriOps-Agent, where remediation plans are evaluated according to the extent to which they are supported by observable system evidence.

A second methodological foundation is provided by structured representations of distributed system behavior. Modern cloud-native applications operate as interconnected collections of services whose states evolve through complex temporal and topological interactions. Joint temporal-structural representation learning demonstrates that combining behavioral dynamics with service dependency structures produces richer descriptions of fault propagation processes across distributed environments [18]. Dependency-aware contrastive learning further highlights the importance of preserving relational information when characterizing operational anomalies and system states [19]. Multiscale anomaly modeling introduces complementary mechanisms for capturing behavioral variations across heterogeneous temporal resolutions and workload conditions [20]. Together, these methodologies establish a representation-centric view of operational intelligence in which system observations are interpreted within broader structural and temporal contexts. VeriOps-Agent adopts this perspective when evaluating diagnostic evidence, enabling evidence assessment to reflect both local observations and system-wide behavioral consistency.

The decision-making process of VeriOps-Agent is further informed by uncertainty-aware operational control methodologies. Risk-calibrated decision frameworks demonstrate how uncertainty estimation can be incorporated into operational policies to support reliability-oriented action selection under dynamic conditions [21]. Credibility-aware self-correction mechanisms similarly illustrate how confidence evaluation can be integrated into downstream decision processes, enabling generated outputs to be assessed before influencing operational actions [22]. Resilience-oriented system evaluation extends this principle by examining system behavior under uncertain and evolving fault scenarios, thereby providing a structured basis for operational reliability assessment [23]. Predictive autoscaling and adaptive orchestration methodologies further contribute mechanisms for incorporating anticipated workload dynamics and future system states into operational decision making [24], [25]. Learning-augmented resource provisioning and long-sequence workload forecasting similarly demonstrate the value of combining historical patterns with predictive system awareness when selecting operational actions [26], [27]. These methodological directions collectively motivate the operation safety and rollback feasibility dimensions of the Evidence Gate, where remediation authorization is formulated as a reliability-oriented decision process rather than a direct consequence of diagnostic inference.

Reliable agent architectures constitute another essential methodological component of the proposed framework. Semantic-aware routing mechanisms demonstrate how intelligent systems can dynamically organize computational pathways according to contextual requirements and task semantics [28]. Freshness-aware knowledge management further highlights the importance of maintaining consistency between reasoning processes and evolving information sources [29]. Memory-centric agent architectures provide mechanisms for organizing long-term operational knowledge through integrated semantic and structural representations, enabling adaptive reasoning over extended time horizons [30]. Efficient model adaptation techniques contribute practical methodologies for aligning model behavior with domain-specific objectives while preserving computational efficiency and deployment scalability [31]. Semantically guided adaptation frameworks similarly demonstrate how domain knowledge can be incorporated into optimization processes to improve controllability and operational alignment [32]. Collectively, these architectural methodologies support a modular design philosophy in which reasoning, memory management, evidence retrieval, adaptation, and execution are organized as coordinated yet distinct functional components. VeriOps-Agent adopts this philosophy through the explicit separation of hypothesis generation, evidence evaluation, and remediation execution.

The broader objective of trustworthy autonomous systems further reinforces the methodological design of VeriOps-Agent. Topology-aware knowledge distillation demonstrates how structural relationships can be preserved across distributed intelligent systems while maintaining efficiency and consistency during knowledge transfer [33]. Privacy-aware adaptation methodologies introduce complementary perspectives on governance, constraint enforcement, and controlled model behavior during deployment and decision execution [34]. These directions collectively support a view of autonomous intelligence in which operational decisions emerge from measurable validation criteria, explicit control mechanisms, and transparent decision pathways. Within VeriOps-Agent, this principle is embodied through a quantitative gating mechanism that transforms generated remediation hypotheses into evidence-verified operational decisions.

III. Methodology

A. Overview

VeriOps-Agent operates through three sequential phases, as illustrated in Fig. 1: (1) Hypothesis Generation, where the LLM produces candidate root causes and remediation plans; (2) Evidence Verification, where each candidate is scored against observable telemetry and operational constraints; and (3) Gated Execution, where a composite gate score determines whether the fix is executed autonomously or escalated to a human operator.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:54 UTC from IEEE Xplore. Restrictions apply.

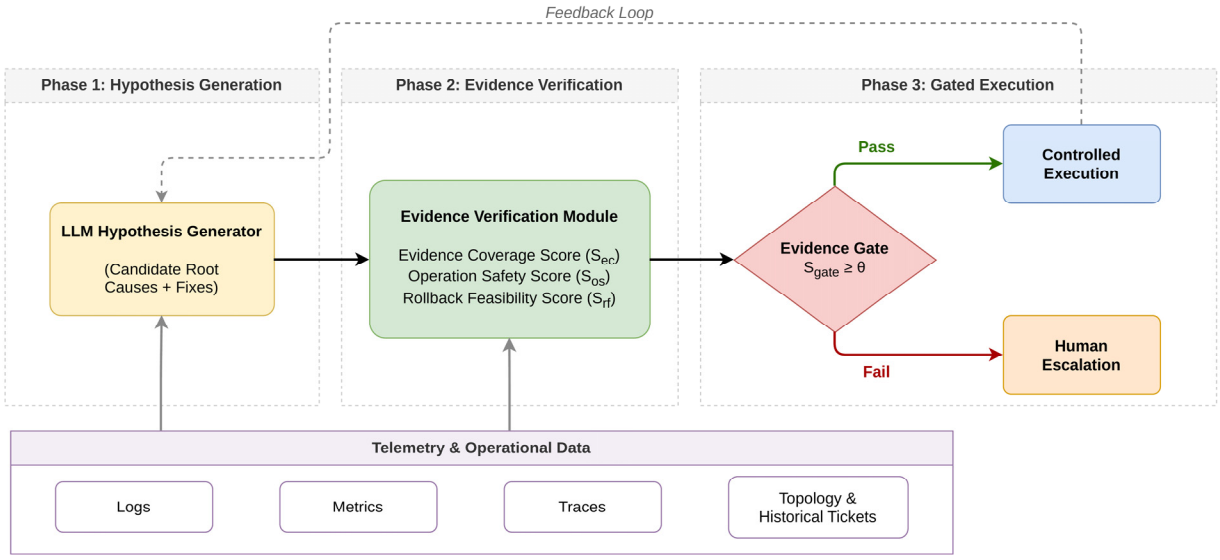


Fig. 1. Architecture of VeriOps-Agent. The LLM generates candidate root causes and remediation plans (Phase 1), which are scored by the evidence verification module along three safety dimensions (Phase 2). The evidence gate then decides between autonomous execution and human escalation (Phase 3). A feedback loop enables iterative refinement upon execution failure.

B. *Phase 1: Hypothesis Generation*

Given an incident alert and associated telemetry data (logs $\mathcal{L}$, metrics $\mathcal{M}$, traces $\mathcal{T}$, topology graph $\mathcal{G}$, and historical tickets $\mathcal{H}$), the LLM agent generates a set of k candidate hypotheses:

$$\{(r_i, f_i)\}_{i=1}^{k} = \text{LLM}(\mathcal{L}, \mathcal{M}, \mathcal{T}, \mathcal{G}, \mathcal{H}; \mathcal{P}) \quad (1)$$

where r_i is a root cause description, f_i is the corresponding remediation plan (a sequence of API calls or shell commands), and $\mathcal{P}$ denotes the prompt template incorporating chain-of-thought reasoning. The LLM operates in a ReAct-style loop, iteratively querying telemetry APIs to collect additional diagnostic data before finalizing its hypotheses. Historical incident tickets are retrieved via RAG to provide few-shot context.

C. *Phase 2: Evidence Verification*

Each hypothesis (r_i, f_i) is evaluated along three dimensions.

Evidence Coverage Score (S_{ec}). This metric measures the proportion of claims in the root cause hypothesis that are corroborated by observable telemetry data:

$$S_{ec}(r_i) = \frac{1}{|C_i|} \sum_{j=1}^{|C_i|} 1\left[\exists\, e \in \mathcal{E}: \text{supports}(e, c_j)\right] \quad (2)$$

where C_i is the set of causal claims extracted from r_i, $\mathcal{E}$ is the collected evidence set, and the supports function determines whether evidence e is consistent with claim c_j through pattern matching and threshold-based anomaly detection.

Operation Safety Score (S_{os}). This score quantifies the safety of the proposed remediation action:

$$S_{os}(f_i) = 1 - \frac{w_b \cdot B(f_i) + w_d \cdot D(f_i)}{w_b + w_d} \quad (3)$$

where $B(f_i) \in [0,1]$ is the normalized blast radius (fraction of dependent services affected), $D(f_i) \in [0,1]$ is the data loss risk, and w_b, w_d are configurable weights.

Rollback Feasibility Score (S_{rf}). This score assesses whether a clear rollback path exists:

$$S_{rf}(f_i) = \alpha \cdot P(f_i) + \beta \cdot R(f_i) + \gamma \cdot T_{\text{feas}}(f_i) \quad (4)$$

where $P(f_i) \in \{0,1\}$ indicates whether a pre-action snapshot is available, $R(f_i) \in \{0,1\}$ indicates whether a rollback procedure is defined, $T_{\text{feas}}(f_i) = 1 - \min(t_{\text{rollback}}(f_i)/t_{\text{SLA}}, 1)$ is the time feasibility factor, and $\alpha + \beta + \gamma = 1$.

D. *Phase 3: Evidence Gate*

The three scores are combined into a composite gate score:

$$S_{gate}(r_i, f_i) = \lambda_1 S_{ec}(r_i) + \lambda_2 S_{os}(f_i) + \lambda_3 S_{rf}(f_i) \quad (5)$$

where $\lambda_1 + \lambda_2 + \lambda_3 = 1$ are weighting parameters. The execution decision is:

$$\text{Decision} = \begin{cases} \text{Execute}(f_{i^*}) & \text{if } S_{gate}(r_{i^*}, f_{i^*}) \geq \theta \\ \text{Escalate to Human} & \text{otherwise} \end{cases} \quad (6)$$

where $i^* = \text{argmax}_i S_{gate}(r_i, f_i)$ and θ is a configurable threshold.

E. *Implementation of Evidence Scoring*

Table I summarizes the computation of each scoring dimension. Causal claims C_i are extracted using structured output parsing. The supports function uses rule-based pattern matching: metric claims are checked against a 2σ anomaly threshold, log claims are matched via regular expressions, and trace claims are verified against baseline p99 values. Blast radius $B(f_i)$ is computed from the service dependency graph, and data loss risk $D(f_i)$ is assigned from a predefined action-type lookup table. Rollback availability is checked against Kubernetes resource history and Helm release records.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:54 UTC from IEEE Xplore. Restrictions apply.

TABLE I. EVIDENCE SCORING IMPLEMENTATION DETAILS.

Score	Input	Method
S_{ec}	Claims + telemetry	Rule-based matching + anomaly thresholds
S_{os}	Action type + dependency graph	Weighted blast radius + data-loss lookup
S_{rf}	Snapshot, runbook, SLA constraints	Binary checks + normalized time ratio

IV. EXPERIMENTAL SETUP

A. Benchmark Environment

We use AIOpsLab to instantiate Kubernetes-based microservice fault environments, leveraging its fault injection, workload generation, and telemetry export capabilities (Prometheus metrics, Jaeger traces, Elasticsearch logs). We adapt SRE-style incident resolution tasks inspired by ITBench for evaluation. The deployed application consists of 12 microservices with realistic inter-service dependencies. Each fault instance is evaluated over three independent runs with different workload seeds; reported rates are averaged across these trials.

B. Fault Scenarios

We evaluate on six categories of injected faults, each with 15 instances (90 total): (1) CPU Throttling—resource limits set below workload requirements; (2) Pod Crash—CrashLoopBackOff due to OOM kills or liveness probe failures; (3) Wrong Environment Variable—misconfigured connection strings or feature flags; (4) Database Connection Failure—connection pool exhaustion or credential rotation errors; (5) Service Timeout—upstream dependency latency exceeding configured timeouts; (6) Configuration Drift—unintended changes to ConfigMaps or resource quotas.

C. Baselines

We compare VeriOps-Agent against four baselines: (1) Direct LLM—GPT-4 directly generates diagnosis and executes remediation without verification; (2) RAG+LLM—GPT-4 augmented with retrieval from a historical incident knowledge base, constructed from validation records with all test scenarios excluded; (3) Runbook—rule-based matching of alert patterns to predefined remediation scripts; (4) No-Gate Agent—a ReAct-style agent with the same tool access as VeriOps-Agent but without the evidence gate.

All LLM-based methods use GPT-4 (gpt-4-0613) with temperature 0.2, a maximum output of 4096 tokens, and a tool-access budget of up to 15 API calls per incident. For VeriOps-Agent, we set $\lambda_1 = 0.4$, $\lambda_2 = 0.3$, $\lambda_3 = 0.3$ and $\theta = 0.55$ based on validation on a held-out set of 18 scenarios (3 per fault type, disjoint from the 90 test scenarios).

D. Metrics

We evaluate using five metrics: Top-1/Top-3 RCA Accuracy (whether the correct root cause appears in the top-1 or top-3 predictions), Wrong Fix Rate (WFR) (percentage of incidents resulting in an incorrect autonomous remediation, with escalated cases remaining in the denominator), Mean Time to Repair (MTTR) (time from detection to resolution, including human handling time), Human Escalation Rate (percentage of incidents escalated to operators), and Token Cost (average LLM tokens consumed per incident).

TABLE II. OVERALL PERFORMANCE COMPARISON ACROSS 90 FAULT SCENARIOS, EACH REPEATED OVER THREE INDEPENDENT RUNS. BEST RESULTS ARE IN BOLD; SECOND-BEST ARE UNDERLINED FOR ACCURACY, WFR, AND MTTR.

Method	Top-1 Acc.	Top-3 Acc.	WFR (%)	MTTR (min)	Esc. (%)
Direct LLM	0.42	0.63	38.4	18.5	0
RAG+LLM	0.51	0.71	30.8	15.2	0
Runbook	0.38	0.38	40.7	**8.3**	22.0
No-Gate Agent	0.57	0.76	34.2	12.1	0
VeriOps-Agent	**0.58**	**0.78**	**17.2**	14.8	25.6

V. RESULTS AND ANALYSIS

A. Overall Performance

Table II presents the overall performance comparison across all 90 fault scenarios. VeriOps-Agent achieves the lowest wrong fix rate (17.2%) among all methods, representing a 55% relative reduction compared to Direct LLM (38.4%) and a 50% reduction compared to the No-Gate Agent (34.2%). This improvement comes from the evidence gate filtering out poorly supported hypotheses before execution.

The RCA accuracy of VeriOps-Agent (Top-1: 0.58, Top-3: 0.78) is comparable to the No-Gate Agent, confirming that the evidence gate does not degrade diagnostic capability. The MTTR (14.8 min) is moderately higher than the No-Gate Agent (12.1 min), reflecting the overhead of evidence verification and human handling time, but this is offset by the substantially lower wrong fix rate.

B. Per-Fault-Type Analysis

Fig. 2 presents a heatmap of wrong fix rates across fault types and methods. VeriOps-Agent demonstrates consistent improvements across all six fault categories. The largest improvement is observed for Wrong Environment Variable faults (12% vs. 30% for Direct LLM and 55% for Runbook), because environment variable misconfigurations produce clear evidence signatures in application logs that the evidence coverage score captures effectively. The smallest improvement is for Database Connection Failure (20% vs. 38% for No-Gate Agent), where the root cause often involves external dependencies whose state is not fully observable through the application's own telemetry.

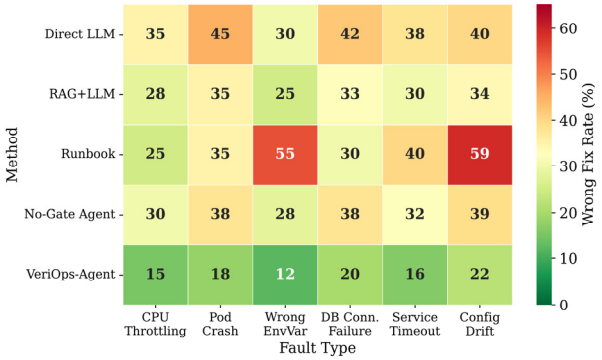


Fig. 2. Heatmap of wrong fix rates (%) across six fault types and five methods. Lower values (green) indicate fewer erroneous remediations. Per-row averages are consistent with the overall WFR in Table II.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:54 UTC from IEEE Xplore. Restrictions apply.

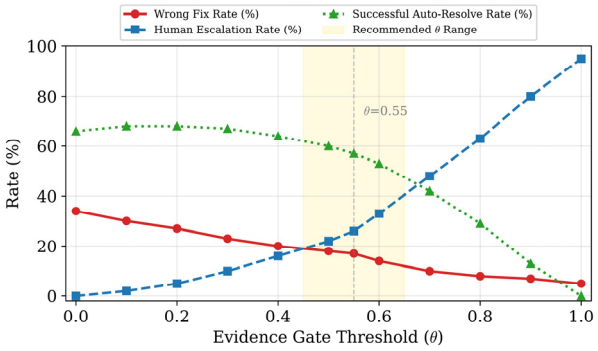


Fig. 3. Effect of evidence gate threshold θ on wrong fix rate, human escalation rate, and successful auto-resolve rate. The shaded region indicates the recommended operating range.

C. *Evidence Gate Threshold Analysis*

Fig. 3 examines the effect of the evidence gate threshold θ. As θ increases from 0 to 1.0, the wrong fix rate decreases from 34% to 5%, while the human escalation rate increases from 0% to 95%. The successful auto-resolve rate peaks near $\theta = 0.1$–0.3 at approximately 68%. The recommended operating range is $\theta \in [0.45, 0.65]$; at $\theta = 0.55$, approximately 57.2% of incidents are resolved correctly and autonomously.

D. *Token Cost and Ablation Study*

VeriOps-Agent consumes an average of 4,850 tokens per incident, compared to 2,100 for Direct LLM, 3,400 for RAG+LLM, and 4,200 for the No-Gate Agent. The additional cost (approximately 15% more than No-Gate Agent) stems from evidence verification queries and structured scoring rationale generation, which we consider acceptable given the substantial reduction in wrong fixes.

Table III presents the ablation study results, where one scoring dimension is removed at a time from the gate score. Removing the evidence coverage score (S_{ec}) causes the largest degradation, increasing the wrong fix rate from 17.2% to 28.3%, confirming that evidence coverage is the most critical safety dimension. Removing operation safety or rollback feasibility has a smaller but still meaningful impact, indicating that all three dimensions contribute to overall safety.

TABLE III. ABLATION STUDY ON EVIDENCE GATE COMPONENTS. WFR: WRONG FIX RATE. ESC.: HUMAN ESCALATION RATE.

Configuration	WFR (%)	Esc. (%)
Full model ($S_{ec} + S_{os} + S_{rf}$)	17.2	25.6
w/o Evidence Coverage (S_{ec})	28.3	18.4
w/o Operation Safety (S_{os})	22.1	20.7
w/o Rollback Feasibility (S_{rf})	20.5	22.3

VI. DISCUSSION AND CONCLUSION

We presented VeriOps-Agent, a framework for safe backend incident remediation that introduces an evidence gate between LLM-based diagnosis and autonomous execution. By scoring candidate fixes along evidence coverage, operation safety, and rollback feasibility dimensions, VeriOps-Agent reduces wrong fix rates by approximately 50–55% compared with the Direct LLM and No-Gate Agent baselines, with moderate overhead in MTTR and token cost.

VeriOps-Agent inherits the diagnostic limitations of the underlying LLM: when the LLM fails to generate the correct root cause, the gate can only prevent incorrect execution by escalating to humans. The framework also assumes sufficient telemetry data for verification; incomplete observability may make the evidence coverage score unreliable. Our evaluation uses synthetic fault injections, and real production incidents exhibit greater complexity. The threshold θ and weight parameters require operational calibration, and the gate can be configured per-service for different risk profiles. Future work includes incorporating causal inference from service dependency graphs and dynamically adapting the gate threshold based on system criticality.

REFERENCES

[1] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proceedings of the 45th IEEE/ACM international conference on software engineering (ICSE)*, 2023, pp. 1737–1749.

[2] Y. Chen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *Proceedings of the nineteenth european conference on computer systems (EuroSys)*, 2024, pp. 674–688.

[3] Z. Wang *et al.*, "RCAgent: Cloud root cause analysis by autonomous agents with tool-augmented large language models," in *Proceedings of the 33rd ACM international conference on information and knowledge management (CIKM)*, 2024.

[4] Y. Jiang *et al.*, "Xpert: Empowering incident management with query recommendations via large language models," in *Proceedings of the 46th IEEE/ACM international conference on software engineering (ICSE)*, 2024.

[5] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proceedings of the 11th international conference on learning representations (ICLR), 2023.

[6] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[7] L. Huang *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *arXiv preprint arXiv:2311.05232*, 2023.

[8] S. Jha, R. Arora, Y. Watanabe, *et al.*, "ITBench: Evaluating AI agents across diverse real-world IT automation tasks," in *Proceedings of the 42nd international conference on machine learning (ICML)*, in PMLR, vol. 267, 2025, pp. 27134–27197.

[9] J. Xu *et al.*, "OpenRCA: Can large language models locate the root cause of software failures?" in *Proceedings of the 13th international conference on learning representations (ICLR)*, 2025.

[10] Y. Chen *et al.*, "AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds," *Proceedings of Machine Learning and Systems (MLSys)*, 2025.

[11] Roy et al., "Exploring LLM-based agents for root cause analysis," in Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion), 2024, pp. 208–219.

[12] L. Zhang et al., "ThinkFL: Self-refining Failure Localization for Microservice Systems via Reinforcement Fine-Tuning," arXiv preprint arXiv:2504.18776, 2025.

[13] S. Lin, Y. Lu, Z. Peng, S. Sun, Y. Wang and M. Wang, "XBRL-Aware LLM Agents for Reliable Financial Statement Analysis: Structured Reasoning and Anti-Hallucination Verification," 2026.

[14] Y. Xing, M. Wang, Y. Deng, H. Liu and Y. Zi, "Explainable Representation Learning in Large Language Models for Fine-Grained Sentiment and Opinion Classification," 2025.

[15] W. Huang, H. Zhuang, W. Huang, J. Kou, R. Wei and N. Lyu, "MERMAIDS: Structured Memory and Evidence Reuse for Reducing Multi-Hop Retrieval Hallucinations in Agentic RAG," in 2026 IEEE 8th

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:54 UTC from IEEE Xplore. Restrictions apply.

International Conference on Communications, Information System and Computer Engineering (CISCE), 2026, pp. 1587–1592.

[16] Q. Deng, S. Liu, H. Qiu, N. Long, W. Lin and S. Chen, “Unified Semantic Space and Attention-Based Fusion Framework for Multi-Source Heterogeneous Retrieval-Augmented Generation,” 2026.

[17] Y. Yang, Z. Liu, C. Chen, S. Dang and K. Wu, “Drift-Aware Retrieval-Augmented Log Anomaly Detection for Cloud AIOps,” 2026.

[18] Y. Xue, Y. Wang, A. Zhu, X. Sun and C. Zhang, “Joint Temporal-Structural Representation Learning for Distributed Fault Discrimination in Microservice Architectures,” arXiv preprint arXiv:2605.01776, 2026.

[19] Y. Xing, Y. Deng, H. Liu, M. Wang, Y. Zi and X. Sun, “Contrastive Learning-Based Dependency Modeling for Anomaly Detection in Cloud Services,” arXiv preprint arXiv:2510.13368, 2025.

[20] Y. Ma, “Anomaly Detection in Microservice Environments via Conditional Multiscale GANs and Adaptive Temporal Autoencoders,” Transactions on Computational and Scientific Methods, vol. 4, no. 10, 2024.

[21] X. Huang, T. Xia, Y. Zhou, J. Liao, L. Ren and F. Chang, “Risk-Calibrated Uncertainty-Aware Edge–Cloud Scheduling for Reliable Multi-Tenant ML Inference,” 2026.

[22] H. Huang, L. Ma, R. Long and L. Mao, “Label Credibility-Aware Self-Correction for Robust Text Classification in Large Language Models,” 2026.

[23] Xue, T. Ying, P. Hu, L. Wang, Z. Qing and T. Ding, “Fuzzing Microservice Resilience under Uncertainty with LLM-Guided Scenario Generation,” 2026.

[24] Y. Hsieh, Z. Xiao, X. Liu, Z. Wang, Y. Liu and X. Li, “An LLM-Augmented Predictive Auto-Scaling Framework for Latency-Sensitive Microservice Backends,” 2026.

[25] Y. Liu, “SlessTrain: Adaptive Serverless-Orchestrated Distributed Training and Inference for Online Machine Learning Workloads,” 2026.

[26] J. Jiang, J. Hu and Y. Lyu, “Learning Augmented Cold Start Mitigation for Serverless Machine Learning Inference,” 2026.

[27] R. Meng, Z. Huang, Y. Yang, K. Zeng, X. Zhang and S. Y. Huang, “Research on Informer Long Sequence Dependency Modeling Method for Short-Term Prediction of Request Volume of Cloud-Native Backend Services,” 2026.

[28] Z. Jiang, S. Zhang, W. Huang, S. Pi, S. Shawulieti and S. Xu, “Semantic-Aware Intelligent Request Routing in Multi-Model LLM Backend Gateways,” 2026.

[29] S. Dang, C. Chen, K. Wu, Z. Liu and Y. Yang, “CacheSense: Freshness-Aware Semantic Caching with Selective Invalidation for LLM-Serving Backends,” 2026.

[30] L. Mao, Z. Gu, R. Long, T. Song and H. Huang, “MemWeave: An Adaptive Framework for Long-Term Agent Knowledge Management via Unified Graph and Vector Memory,” 2026.

[31] Z. Huang, X. Zhang, Y. Yang, K. Zeng, R. Meng and S. Y. Huang, “Resource-Efficient Fine-Tuning of Large Language Models via Subspace Compression and Sparse Updates,” 2026.

[32] H. Zheng, Y. Ma, Y. Wang, G. Liu, Z. Qi and X. Yan, “Structuring Low-Rank Adaptation with Semantic Guidance for Model Fine-Tuning,” in 2025 6th International Conference on Electronic Communication and Artificial Intelligence (ICECAI), 2025, pp. 731–735.

[33] K. Wu, J. Qiu, Y. Xue, Z. Yang, J. Sun and Y. Wang, “Topology-Aware Cross-Modal Distillation for Communication-Efficient Decentralized Multimodal Federated Learning,” 2026.

[34] Y. Li, “Task-Aware Differential Privacy and Modular Structural Perturbation for Secure Fine-Tuning of Large Language Models,” Transactions on Computational and Scientific Methods, vol. 4, no. 7, 2024.

Authorized licensed use limited to: UNIVERSIDADE DO VALE DO RIO DOS SINOS. Downloaded on October 28,2026 at 02:37:54 UTC from IEEE Xplore. Restrictions apply.